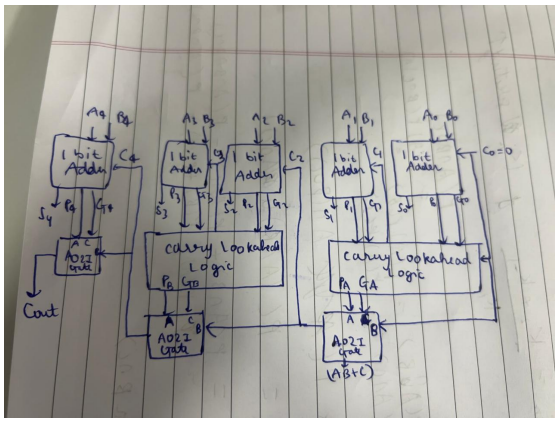
VLSI Project

Subal Manchanda : 2024102033

December 2025

1 Proposed Structure

I have implemented a 5-bit CLA adder implementation in which we have a Carry-Lookahead Logic broken into 3 parts (2-2-1). I have used AO21 gates for optimising delay along the critical path and also used traditional CMOS gates like AND, NOT. Also, I have used D-Flip Flops for gatekeeping inputs and outputs to ensure proper timing in a real life application. This adder implementation does not use more than 2-input AND Gates which leads to a less fan-in. The block diagram of the circuit is given below:



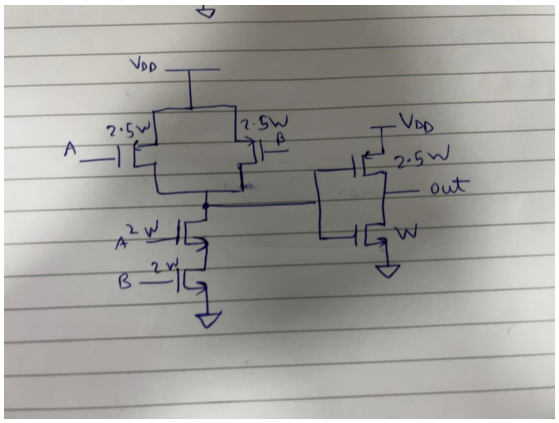
2 Design details of each block

Taking W = 10 * LAMBDA

Gates:

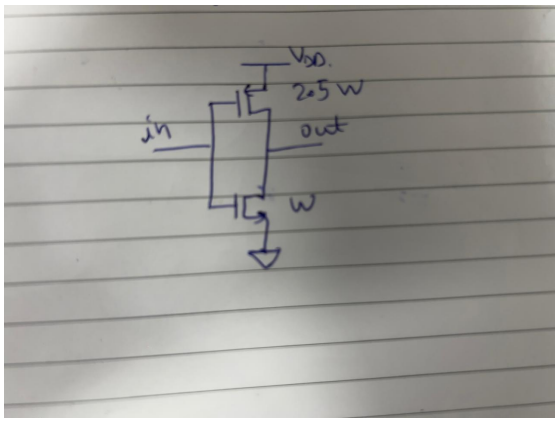
2.1 AND GATE

This has been implemented using static CMOS logic with sizing as shown in the figure.



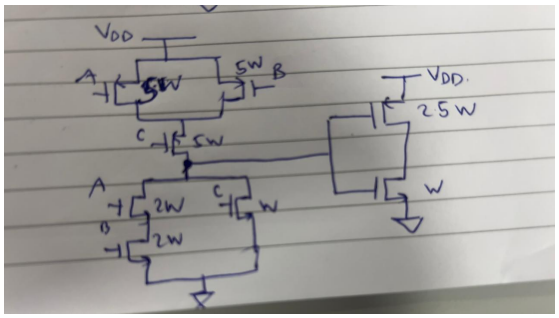
2.2 NOT GATE

This has been implemented using static CMOS logic with sizing as shown in the figure.



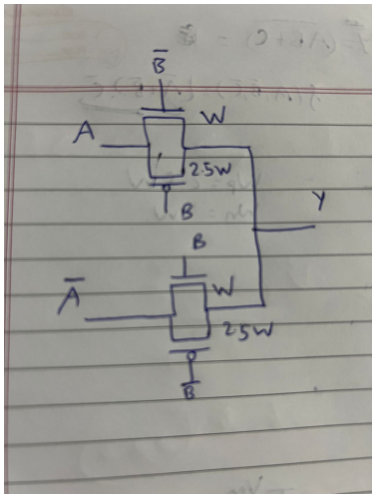
2.3 AO21 Gate

This has been implemented using static CMOS logic with sizing as shown in the figure.



2.4 XOR Gate

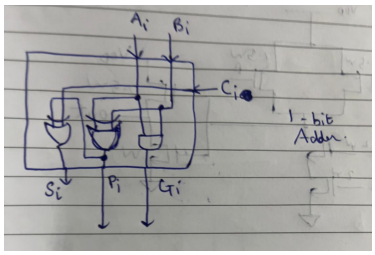
This has been implemented using Transmission gate logic with sizing as shown in the figure.



Different blocks in the circuit:

2.5 1-bit Adder

The adder block has been implemented as follows using the gates we described above:



The adder block provides the propogate and generate bit, along with computing the sum bit as follows:

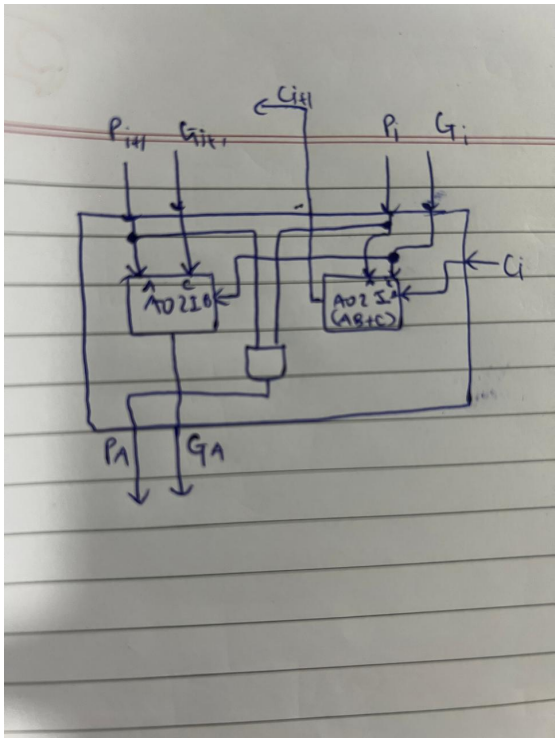
$$P = A \oplus B$$

$$G = A \cdot B$$

$$S = P \oplus C_{\text{in}}$$

2.6 Carry Lookahead Logic (CLL) Block

The CLL block has been implemented as follows using the gates we described above:



The CLL block implements the following logics:

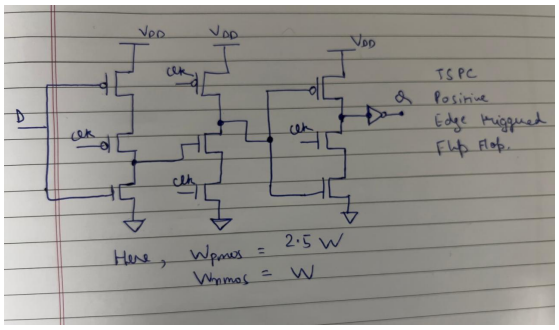
$$C_{i+1} = G_i + P_i C_i$$

$$P_A = P_{i+1} P_i$$

$$G_A = G_{i+1} + P_{i+1} G_i$$

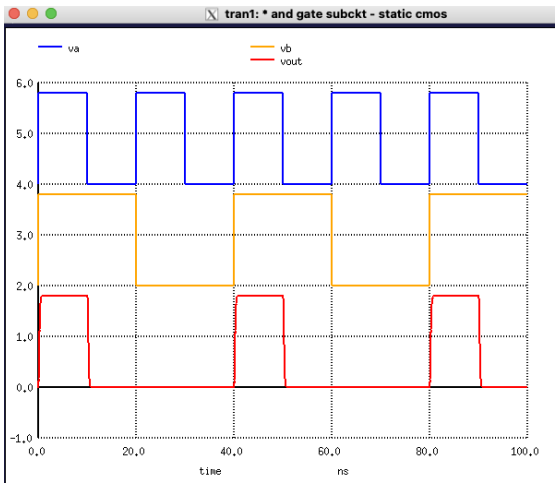
2.7 D Flip-Flop

This has been implemented using TSPC logic style as shown in the figure with sizing shown:

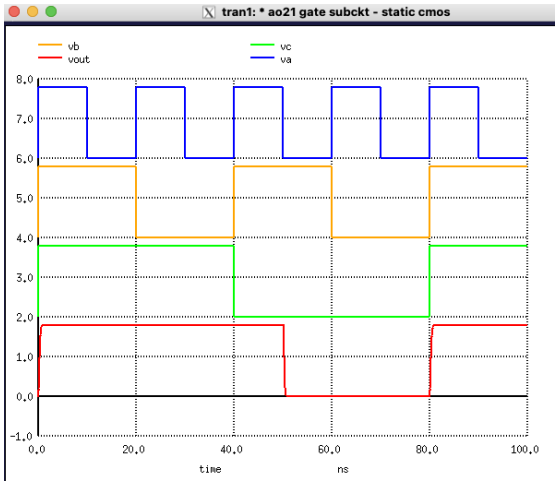


3 Simulating Each Gate seperately:

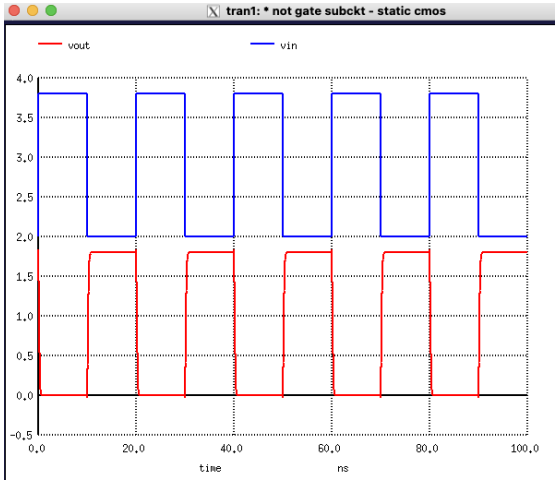
3.1 AND GATE



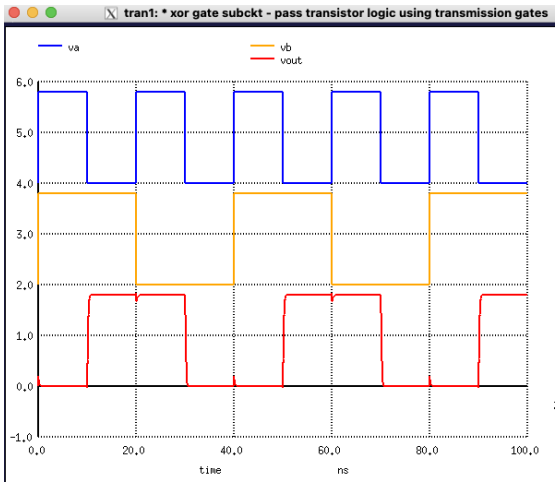
3.2 AO21 GATE



3.3 NOT GATE

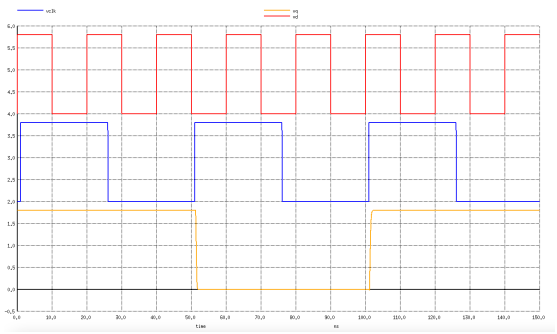


3.4 XOR GATE



4 Setup Time, Hold Time, Clock to Q Delay for D-Flip Flop

First, we can observe that the D-Flip Flop works as intended.



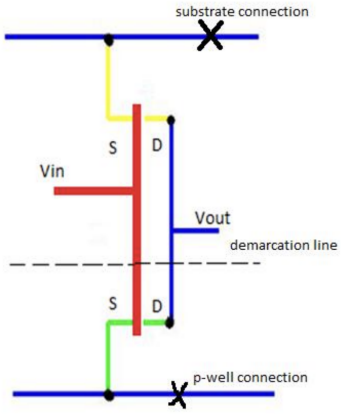
Then, note that we can argue that the hold time would be approximately 0 since after the positive edge of the clock, the output of gets latched and any change in input would not affect the output.

Then, for the setup time we can find the propogation time of input to the intermediate node X. For Clock to propogation delay, we measured for both cases when the output is rising and when the output is falling. We have obtained these value from simulation as follows:

```
No. of Data Rows : 15144
Warning: Unable to load any usable fontset
t_setup            =  5.372911e-11 targ=  5.006873e-08 trig=  5.001500e-08
t_clk_to_fall_q    =  3.972456e-10 targ=  5.140225e-08 trig=  5.100500e-08
t_clk_to_rise_q    =  3.492634e-10 targ=  1.013543e-07 trig=  1.010050e-07
```

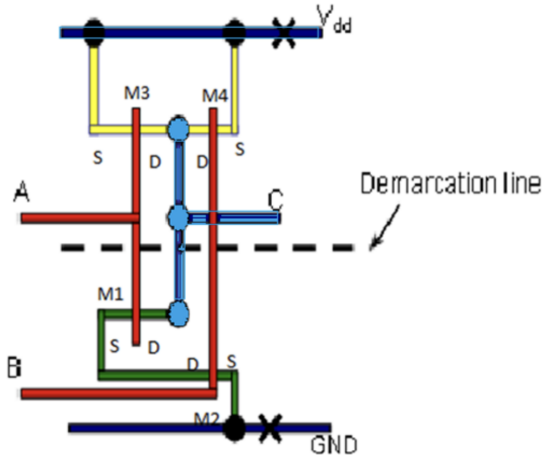
5 Stick diagrams for all gates

5.1 NOT GATE

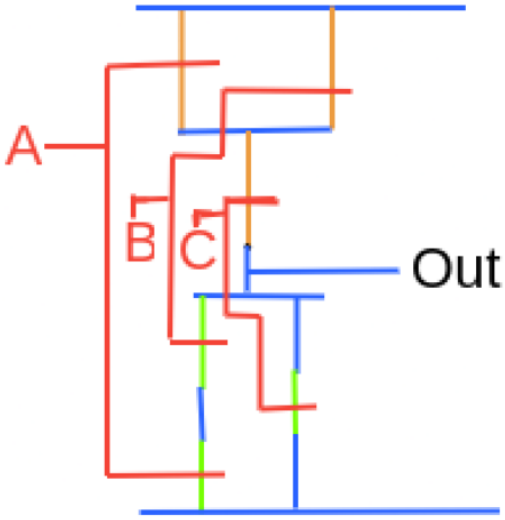


5.2 AND GATE

Since CMOS Implementation of AND Gate is just the CMOS of NAND followed by inverter, we are giving here the stick diagram for NAND which can be cascaded with an inverter (shown above) to obtain the AND function:

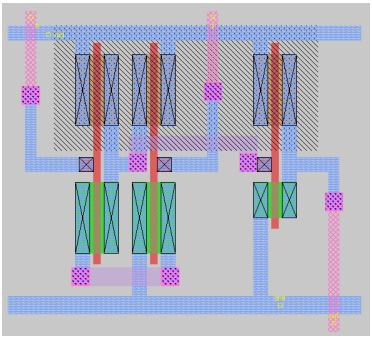


5.3 AO21 Gate

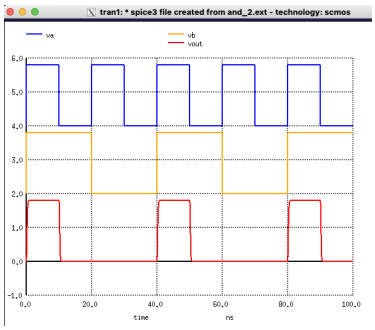


6 Layout for each block

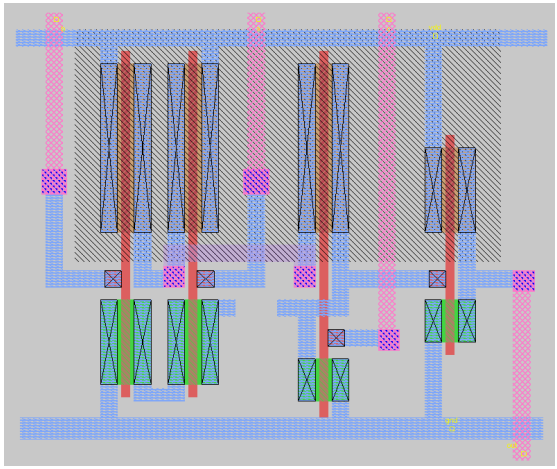
6.1 AND GATE



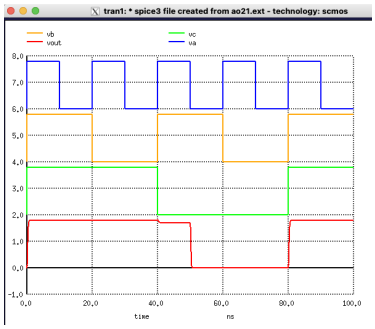
Post-Layout Simulation:



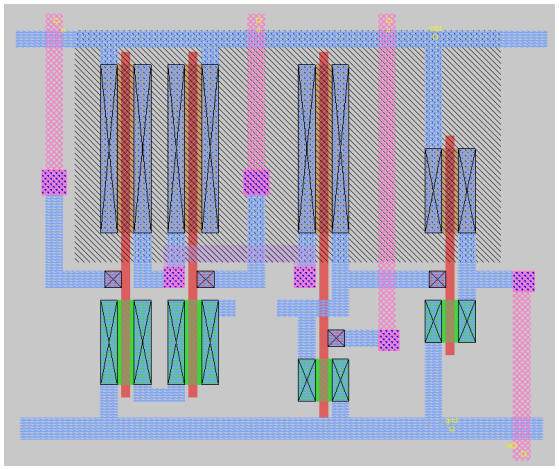
6.2 AO21 GATE



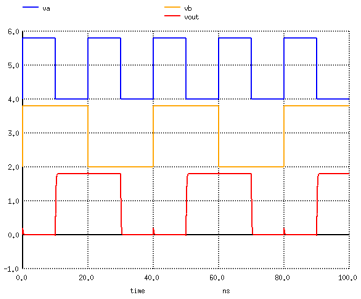
Post-Layout Simulation:



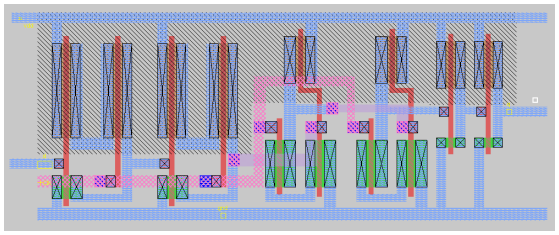
6.3 XOR GATE



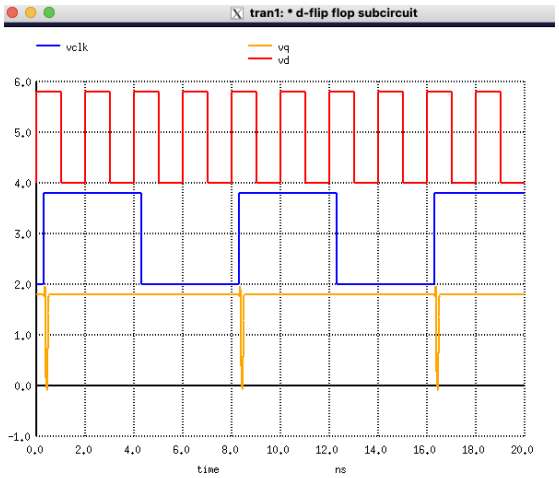
Post-Layout Simulation:



6.4 D Flip Flop



Post-Layout Simulation:



Using these basic gates, we can implement each part of the circuit as discussed in Section 2.

Comparison of Post-Layout and Pre-Layout Results:

From the waveforms, the following observations can be made:

- **Propagation delay (t_{pd}):** Post-layout v_{out} shows a noticeably larger delay during both rising and falling transitions due to interconnect parasitics (extracted R and C). Thus,

$$t_{pd,\text{post}} > t_{pd,\text{pre}}.$$

- **Rise/Fall time degradation:** The post-layout output transition is slower and less steep compared to the pre-layout case, indicating increased RC loading:

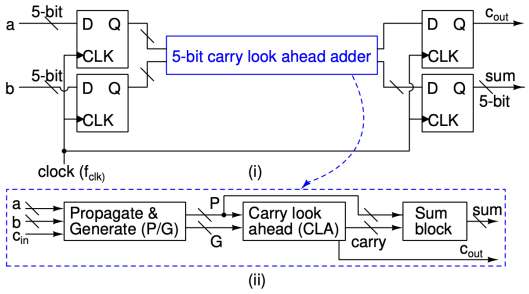
$$t_{r,\text{post}} > t_{r,\text{pre}}, \qquad t_{f,\text{post}} > t_{f,\text{pre}}.$$

- **Final logic levels:** Both pre- and post-layout waveforms reach the correct logic levels, but the post-layout output settles slightly later.

- **Overall effect of layout parasitics:** The post-layout waveform shows delayed edges, longer transition times, and slightly increased settling time. Functionality is preserved, but timing is degraded.

7 Integrating the entire circuit

We will use the 5-bit CLA adder shown in figure 1, along with Flip Flops at inputs and outputs (for gatekeeping) to observe the functioning of the entire circuit.



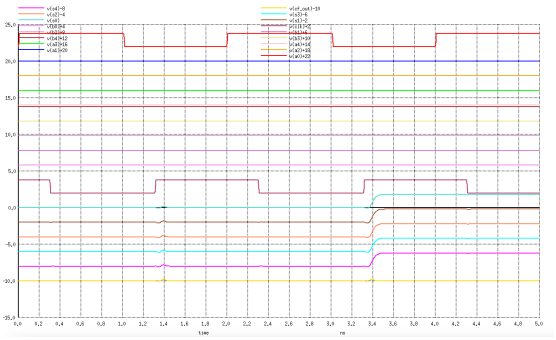
7.1 Finding the Worst Case Delay of the adder circuit

One of the possibilities for the worst case delay of the adder will occur when input numbers are a = 00000 and b = 11111.

This is because in such a case $P_A = 1$, $G_A = 0$ and $P_B = 1$, $G_B = 0$ which means S_4 will have to wait for C_4 which inturn will have to wait for C_2 to propogate.

In such a case, we obtain worst case delay. Showing the output of the circuit for this : (Note that the first rising edge does not produce an output, which is

because of the fact that the inputs get loaded to the adder in the first rising edge of the clock, thus we start observing output from the second rising edge). As we can clearly see, when a=00000, b=11111 sum = 11111, cout = 0.



Note that the delay obtained here from the rising edge of the clk to s4 is the worst case delay of the 5-bit adder.

From simulation results, we obtain:

```
 Reference value :  4.44001e-09
No. of Data Rows : 576
Warning: Unable to load any usable fontset
t_delay            =  8.422251e-11 targ=  3.399223e-09 trig=  3.315000e-09
```

Delay of 5-bit adder = 84.22ps.

7.2 Minimum Clock Period

From timing diagram analysis, we know:

$$t_{cq} + t_d \leq T_{clk} - t_{su}$$

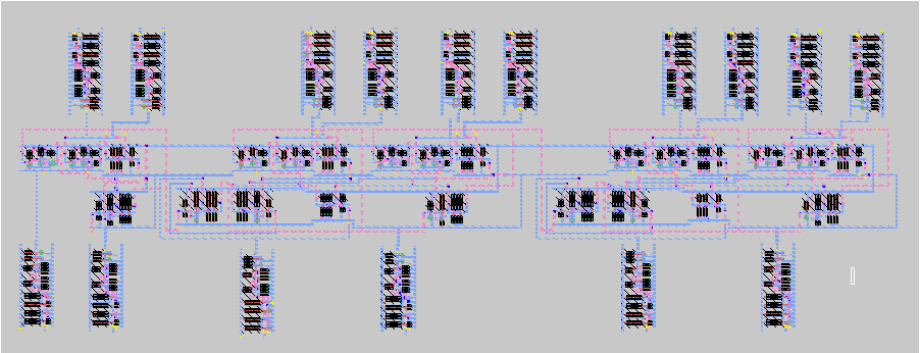
$$T_{clk}^{\min} = t_{cq-max} + t_d + t_{su}$$

As obtained in Section 4: Using $t_{su} = 0.053$ns, $t_{cq-max} = 0.39$ns, and $t_d = 0.084$ns: We obtain

$$T_{clk}^{\min} = 0.534ns$$

Maximum Clock Frequency = 1.87GHz

8 Floor Plan and Layout for the Complete Circuit



Horizontal and Vertical Pitch in VLSI Layout

- **Pitch** is the center-to-center spacing between two repeating layout features.
- **Horizontal pitch** is the spacing between features along the x-direction.
- **Vertical pitch** is the spacing between features along the y-direction.

Pitch is used to define the routing grid, standard-cell row spacing, and the regularity of repeated structures in a floorplan or layout.

9 Post-Layout Integrated Design

Using the extracted netlist, we have obtained the worst case delay of the adder to be: (Taking input a=00000 b=11111)

```
 Reference value :  1.19673e-08
No. of Data Rows : 2050
Warning: Unable to load any usable fontset
Estimated Delay = 95.3ps
```

10 Maximum Clock Frequency

By using static timing analysis, we may obtain:

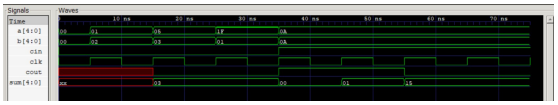
$$t_{cq} + t_d \leq T_{clk} - t_{su}$$

$$T_{clk}^{\min} = t_{cq-max} + t_d + t_{su}$$

Then, we obtain for $t_{su} = 0.053$ns, $t_{cq-max} = 0.39$ns, and $t_d = 95.3$p, Maximum Clock Frequency = 1.85GHz

11 Verilog Description of the 5-bit adder

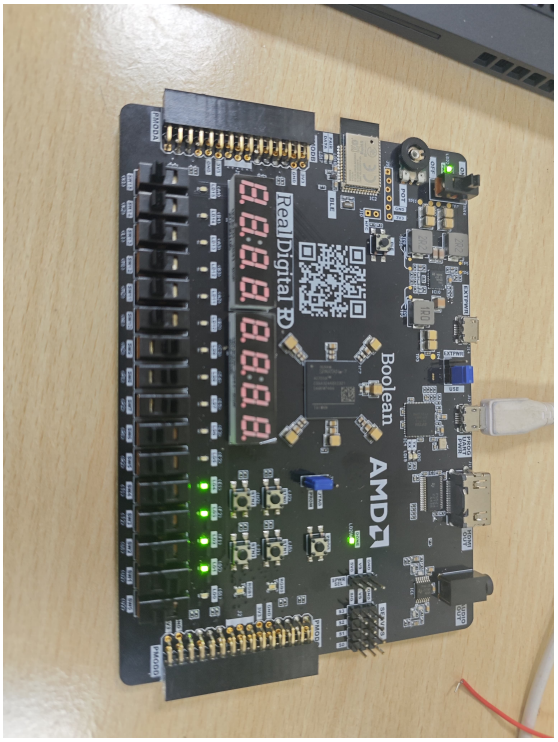
The verilog code has been attached in the pdf, and one of the outputs obtained in gtkwave is shown:

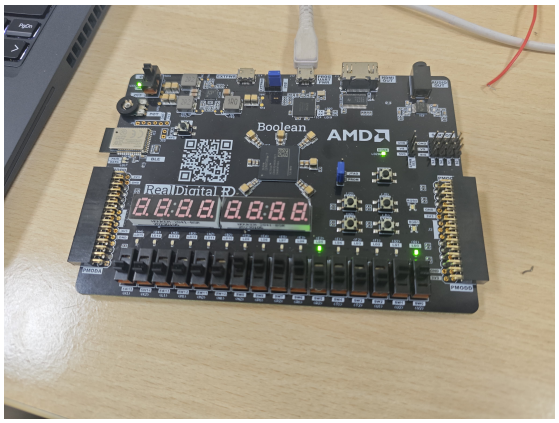


As observable, it takes one rising edge of the clock for the inputs to reach the adder, and it is the second rising edge at whcich we observe the output to the adder circuit.

12 FPGA Simulations

12.1 FPGA Verification of 5-bit Adder





12.2 Oscilloscope Waveform